

WEEK 10 – MERN MINI PROJECT

Student Registration CRUD Application

1. AIM

To develop a Student Registration CRUD Application using the MERN stack (MongoDB, Express.js, React.js, and Node.js) to register student details, display stored records, and perform Create, Read, Update, and Delete operations.

2. ALGORITHM

1. Create a React frontend using Vite and design a registration form with Name, Email, Password, Gender, Country, and Languages fields.
2. Create the backend using Node.js and Express.js. Define REST API routes to register, retrieve, update, and delete student records.
3. Connect Express.js to MongoDB using Mongoose and use the studentdb database with the studentscollection.
4. Start MongoDB, open a terminal in the backend folder, and run **npm start** to start the Express server on port 5000.
5. Open a second terminal in the frontend folder and run **npm run dev**. Open the local URL shown by Vite in a browser.
6. Enter student details and click Register. Check the table, then test Edit, Delete, and Reset to verify the CRUD workflow.

3. PROGRAM

The application uses three main files. The backend code below implements the required REST API. Use the completed App.jsx and App.css files in the frontend folder for the registration form, Fetch API calls, table, Edit/Delete buttons, and styling.

A. Backend – server.js

```
import express from "express";
import mongoose from
"mongoose"; import cors from
"cors";

const app = express();
app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/studentd
b") .then(() => console.log("MongoDB connected"))
.catch(error => console.log("MongoDB error:",
error.message));

const studentSchema = new
mongoose.Schema({ name: String,
email: String, password: String,
gender: String, country: String,
languages: [String] });
```

```

const Student = mongoose.model("Student",
studentSchema);

app.get("/", (req, res) => {
res.send("Express server is working");
});

app.get("/students", async (req, res)
=> { const students = await
Student.find(); res.json(students); });

app.post("/students", async (req, res) => {
const student = new Student(req.body); await
student.save(); res.json({ message:
"Registration successful" }); });

app.put("/students/:id", async (req, res) => { await
Student.findByIdAndUpdate(req.params.id,
req.body); res.json({ message: "Student updated
successfully" }); });

app.delete("/students/:id", async (req, res) => {
await Student.findByIdAndDelete(req.params.id);
res.json({ message: "Student deleted successfully"
}); });

app.listen(5000, () => {
console.log("Server running on port
5000"); });

```

B. Frontend – App.jsx and App.css

App.jsx uses `useState()` to maintain form data and `useEffect()` to load student records. The Fetch API sends

GET, POST, PUT, and DELETE requests to `http://localhost:5000`. The form contains Name, Email, Password, Gender, Country, and Languages controls. App.css styles the form, buttons, messages, and Registered Students table.

4. OUTPUT

Backend terminal (expected messages):

```

Server running on port 5000
MongoDB connected

```

Browser test:

```

http://localhost:5000/ -> Express server is working http://localhost:5173/ ->
Student Registration page

```

Student Registration

Name

Email

Password

Gender

☒ Male ☐ Female

Country

Select Country



Languages

☒ English ☐ Hindi ☐ Telugu

Register

Reset

Student deleted successfully

Registered Students

Name	Email	Gender	Country	Languages	Actions
------	-------	--------	---------	-----------	---------

Student Registration

Name

Email

Password

Gender

☒ Male ☐ Female

Country

Select Country 

Languages

☒ English ☒ Hindi ☒ Telugu

Register

Reset

Registration successful

Registered Students

Name	Email	Gender	Country	Languages	Actions	
janu	janu@gmail.com	Female	India	Telugu, English	<button>Edit</button>	<button>Delete</button>
vani	vani@gmail.com	Female	India	English, Telugu	<button>Edit</button>	<button>Delete</button>

Student Registration

Name

Email

Password

Gender

☐ Male ☐ Female

Country

Select Country

Languages

☐ English ☐ Hindi ☐ Telugu

Register

Reset

Student deleted successfully

Registered Students

Name	Email	Gender	Country	Languages	Actions
vani	vani@gmail.com	Female	India	English, Telugu	Edit Delete

Steps to Execute

1. Start MongoDB Server.
2. In Terminal 1, run: cd backend, then npm start.
3. In Terminal 2, run: cd frontend, then npm run dev.
4. Open the local URL printed by Vite, usually <http://localhost:5173/>.
5. Register a test student and verify the record in the table and in MongoDB Compass.
6. Test Edit, Delete, and Reset.

5. RESULT

Thus, the Student Registration CRUD Application was developed using the MERN stack. The application allows student details to be registered and stored in MongoDB, retrieved and displayed in a table, updated through Edit, and removed through Delete. The Reset button clears the registration form.

